CSE 6389 Special Topics in Advanced Multimedia, Graphics, & Image

RNN for AD classification using fMRI signal

Fall 2025 Project 3

Instructor: Dr. Dajiang Zhu

Submitted By: Srinivasa Sai Abhijit Challapalli.

1002059486

1. Abstract

This report describes my implementation for Programming Assignment 3 (PA3), which focuses on binary classification of Alzheimer's Disease (AD) versus Cognitively Normal (CN) subjects using resting-state functional MRI (fMRI) time-series. The goal is to

(1) implement sequence models inspired by Li & Fan (2019) and Nguyen et al. (2020),

(2) use appropriate data pre-processing and augmentation, and

(3) evaluate model performance under subject-level cross-validation.

I implemented two recurrent architectures:
• A Li-style stacked LSTM with sliding-window data augmentation.
• A Nguyen-style MinimalRNN that directly models subject-level fMRI sequences.

The models are trained and evaluated on a small, curated dataset of 20 subjects, with 5-fold stratified cross-validation at the subject level. Performance is summarized using accuracy, precision, recall, F1-score, and confusion matrices.

2. Dataset Description

The directory structure of the dataset is organized by subject and diagnosis:

```
.
├── AD1/
│   └── fmri_average_signal/
│       ├── raw_fmri_feature_matrix_1.txt
│       ├── raw_fmri_feature_matrix_2.txt
│       └── ...
├── AD2/
│   └── fmri_average_signal/
│       └── raw_fmri_feature_matrix_*.txt
├── ...
├── AD10/
├── CN1/
│   └── fmri_average_signal/
│       └── raw_fmri_feature_matrix_*.txt
├── ...
└── CN10/
```

AD1, AD2, …, AD10 → Alzheimer's Disease subjects (label = 1)
CN1, CN2, …, CN10 → Cognitively Normal controls (label = 0)

Each subject folder contains a subdirectory named fmri_average_signal with multiple text files called raw_fmri_feature_matrix_*.txt. Each file corresponds to one time point and stores a 1D feature vector of region-of-interest (ROI) activations for that time point.

After loading the data, the preprocessed tensor has shape (N, T, R):
• N = 20 subjects (10 AD, 10 CN)
• T = 101 time points per subject
• R = 150 ROIs per time point

This representation is natural for recurrent neural networks: each subject is treated as a multivariate time series of length T with R channels.

3. Pre-processing and Sliding-window Augmentation

3.1 Per-subject z-scoring

For each subject, I applied per-subject z-scoring across time for every ROI. Given a subject's time series $X_{subj} \in \mathbb{R}^{T \times R}$, I compute:
• μ_r = mean over time of ROI r
• σ_r = standard deviation over time of ROI r

Then each time-series entry is normalized as:

$$X_{norm}[t, r] = (X_{subj}[t, r] - \mu_r) / (\sigma_r + \varepsilon)$$

where ε is a small constant for numerical stability. This removes subject-specific scale differences and focuses the models on relative temporal fluctuations.

3.2 Sliding-window augmentation (Li-style)

To follow the spirit of Li & Fan (2019), I implemented sliding-window data augmentation at the sequence level. For each subject's normalized time series $X_{subj} \in \mathbb{R}^{T \times R}$, I generate overlapping windows of fixed length L with stride S:

• Window size L = 40 time points
• Stride S = 20 time points

For T = 101, this yields windows starting at time indices: 0, 20, 40, and 60. Each window is of shape (L, R) = (40, 150), and inherits the subject's label (AD or CN). This effectively

turns each subject into multiple training sequences while preserving the subject-level label.

During evaluation, predictions are made at the window level, and then aggregated back to subject-level using averaged logits (see Section 6).

4. Model Architectures

4.1 Li-style stacked LSTM classifier

The first architecture is a stacked LSTM inspired by Li & Fan (2019), adapted to the small-sample setting of this assignment.

Input: A window-level sequence of shape (L, R) = (40, 150), treated as length L with input_dim = 150.

Architecture:
1. LSTM layer 1
   - input_size = 150
   - hidden_size = 64
   - num_layers = 1
   - batch_first = True

2. LSTM layer 2
   - input_size = 64
   - hidden_size = 32
   - num_layers = 1
   - batch_first = True

3. Sequence pooling
   - Take the last hidden state h_T from the second LSTM (shape: 32). This is used as a compact representation of the window's temporal dynamics.

4. MLP classification head
   - Fully connected: 32 → 16 with ReLU
   - Fully connected: 16 → 1 logit

The model outputs a single logit per window, which is later converted to a probability by

applying a sigmoid. The use of two stacked LSTMs increases model capacity to capture complex temporal patterns while remaining tractable with our limited data.

4.2 Nguyen-style MinimalRNN classifier

The second architecture is a MinimalRNN inspired by Nguyen et al. (2020), designed to provide a simpler recurrent update with fewer gates than a full LSTM.

Input: A full subject-level sequence of shape (T, R) = (101, 150) (i.e., no windowing in the model definition; windowing is handled by the dataset class).

Update equations per time step t:
1. Candidate state:
$z_t = \tanh(W_x x_t + b_x)$

2. Update gate:
$u_t = \sigma(W_h h_{t-1} + V z_t + b_u)$

3. Hidden state update:
$h_t = u_t \odot h_{t-1} + (1 - u_t) \odot z_t$

where $h_t \in \mathbb{R}^{H}$, $z_t \in \mathbb{R}^{H}$, and H is the hidden_dim.

In my implementation:
- input_dim = 150
- hidden_dim = 64

After iterating over all time steps, the final hidden state h_T is passed through the same style of MLP head as the LSTM:
- Fully connected: 64 → 16 with ReLU
- Fully connected: 16 → 1 logit

This MinimalRNN is conceptually simpler than an LSTM but still capable of modeling long-term dependencies through the update gate u_t.

5. Training Procedure and Cross-validation

Both models are trained with the same overall procedure to enable a fair ablation comparison.

Cross-validation:
• Stratified 5-fold cross-validation at the subject level.
• In each fold, 16 subjects are used for training and 4 subjects for validation.
• Stratification preserves the AD/CN proportion in each fold.

Optimization:
• Loss: Binary Cross-Entropy with Logits (BCEWithLogitsLoss in PyTorch).
• Optimizer: Adam with learning rate 1e-3.
• Batch size: 8 windows per batch.
• Number of epochs: 50 per fold.

Model selection per fold:
• At the end of each epoch, I compute subject-level metrics on the validation set.
• I track the epoch with the highest validation F1-score.
• The confusion matrix and metrics reported for each fold correspond to this best-F1 epoch.

5.1 Subject-level evaluation from window predictions

Because the training is window-based, but labels are defined at the subject level, I aggregate window-level outputs back to subject-level predictions as follows:

1. For each subject in the validation set, collect all logits from the windows generated for that subject.
2. Average the logits for that subject: mean_logit = mean_t(logit_t).
3. Convert the mean_logit to a probability via the sigmoid function: $p = \sigma(\text{mean\_logit})$.
4. Threshold at $p \geq 0.5$ to predict AD (1); otherwise, CN (0).

Using these subject-level predictions, I compute:
• Accuracy
• Precision
• Recall
• F1-score
• Confusion matrix (2×2, with CN and AD as classes).

6. Experimental Results and Ablation Study

I ran 5-fold stratified cross-validation for both models. Below I summarize the cross-validated mean and standard deviation across folds, as printed by the training script.

MinimalRNN (Nguyen-style):
- Accuracy : 0.600 ± 0.122
- Precision: 0.633 ± 0.194
- Recall : 0.800 ± 0.245
- F1-score : 0.660 ± 0.095

Li-style LSTM:
- Accuracy : 0.900 ± 0.200
- Precision: 0.900 ± 0.200
- Recall : 1.000 ± 0.000
- F1-score : 0.933 ± 0.133

Overall, the stacked LSTM substantially outperforms the MinimalRNN under these settings, especially in terms of accuracy and F1-score. However, because the dataset is extremely small (only 20 subjects), these numbers should be interpreted with caution; high variance and potential overfitting are expected.

6.1 Ablation table: MinimalRNN vs. LSTM

Model	Accuracy (mean ± std)	Precision (mean ± std)	Recall (mean ± std)	F1-score (mean ± std)
MinimalRNN (Nguyen-style)	0.600 ± 0.122	0.633 ± 0.194	0.800 ± 0.245	0.660 ± 0.095
Stacked LSTM (Li-style)	0.900 ± 0.200	0.900 ± 0.200	1.000 ± 0.000	0.933 ± 0.133

This ablation isolates the effect of the recurrent architecture while keeping the data pre-processing, sliding-window augmentation, optimizer, and training schedule as consistent as possible between models.

7. Discussion

The experimental comparison suggests that the Li-style stacked LSTM is better suited for this small fMRI dataset than the MinimalRNN implementation. Several factors likely contribute to this difference:

1. Model capacity and expressiveness
   • The stacked LSTM has more parameters and richer gating mechanisms (input, forget, and output gates), which may help capture subtle temporal patterns in the ROI time series.
   • The MinimalRNN uses a single update gate and a simpler state update, which can be advantageous for regularization but may underfit complex dynamics in this setting.

2. Sliding-window representation
   • Both models benefit from windowed sequences, but the LSTM seems to leverage the window-level representation more effectively, leading to higher subject-level F1-scores.

3. Overfitting and small-sample effects
   • The dataset has only 20 subjects. Even with data augmentation, the effective number of independent samples remains small.
   • The near-perfect recall (1.0) for the LSTM indicates that the model often correctly identifies AD subjects, but the high variance across folds suggests that the model could overfit particular splits.

4. Limitations
   • No separate external test set is available; all evaluation is done via cross-validation.
   • The models currently treat ROI signals as flat features; future work could incorporate brain network structure (e.g., graph-based methods) for more interpretable modeling.

8. Conclusion

In this assignment, I implemented two recurrent neural network architectures for classifying AD versus CN from resting-state fMRI time-series:
• A Li-style stacked LSTM with sliding-window data augmentation.
• A Nguyen-style MinimalRNN with a simpler recurrent update.

Using per-subject z-scoring, sliding-window augmentation (L = 40, stride = 20), and subject-level 5-fold stratified cross-validation, the stacked LSTM achieved stronger performance, with a cross-validated F1-score of approximately 0.93 compared to 0.66 for the MinimalRNN.

Although the results are promising, they must be interpreted in the context of the very small sample size and potential overfitting. Nevertheless, the experiment demonstrates how different recurrent architectures and data augmentation strategies impact classification performance on fMRI-based AD detection.

This pipeline can be extended to larger datasets, multi-class settings, multimodal inputs, or graph-based representations to better capture the rich structure of brain connectivity.

9. References

[1] Li, H., & Fan, Y. (2019). Interpretable, highly accurate brain decoding of subtly distinct brain states from functional MRI using intrinsic functional networks and long short-term memory recurrent neural networks. NeuroImage, 202, 116059.

[2] Nguyen, M., Sun, N., Alexander, D.C., Feng, J., & Yeo, B.T.T. (2020). Predicting Alzheimer's disease progression using deep recurrent neural networks. NeuroImage, 222, 117203.

The project is uploaded to GitHub:

GitHub Link